IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

X62 - Optimisation pour l'aide à la décision
TP n° 1
Algorithmes génétiques

L'objectif de ce TP est de s'intéresser à la résolution du problème du voyageur de commerce, de manière exacte par force brute sur les cas simples (peu de villes) puis de manière approchée avec un algorithme génétique comme vu en cours.

Le problème du voyageur peut être exprimé de la manière suivante :

> Un représentant de commerce part de sa ville d'origine et doit passer visiter ses clients dans des villes différentes, une fois et une seule. Il a bien sûr intérêt à minimiser la longueur du chemin qu'il va parcourir et il souhaite rentrer dans sa ville d'origine à la fin de son parcours.

Autrement dit, il consiste à trouver, dans un graphe hamiltonien (pondéré), un cycle hamiltonien de longueur minimale.

1 Représentation d'un cycle

Les n villes du problème sont modélisées par un entier entre 1 et n. Un cycle est représenté par un vecteur d'entiers représentant l'ordre de visite des villes (on ne répète pas la dernière ville qui correspond toujours à la première puisqu'il s'agit d'un cycle). Un exemple est donné en figure 1.

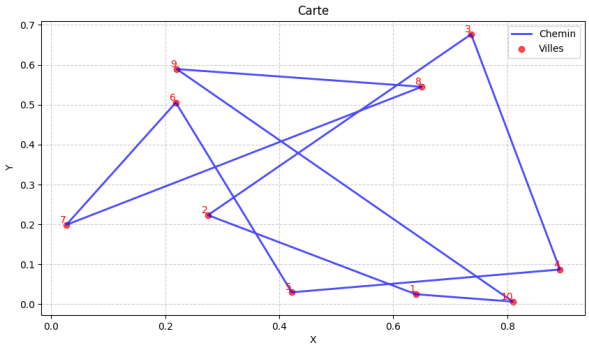

FIGURE 1 – Exemple de représentation pour le cycle $(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$ pour un tirage aléatoire de $n = 10$ villes

Question 1 - Écrire la fonction

```
def init_cities(n):
```

qui crée `n` villes dont les coordonnées (x, y) sont tirées aléatoirement entre 0 et 1.

Question 2 - Écrire la fonction

```
def draw_tour(cities, tour):
```

qui affiche la position des villes (`cities` une liste de coordonnées) sur un graphique et un cycle (`tour`) si le paramètre n'est pas vide (voir figure ci-dessus).

Longueur d'un cycle

L'ensemble des algorithmes que nous allons développer se basent sur le calcul d'une distance entre villes. Afin de ne pas effectuer les calculs plusieurs fois, il peut être utile d'utiliser une matrice contenant l'ensemble des distances entre toutes les villes deux à deux.

Question 3 - Écrire la fonction

```
def distance(city1, city2):
```

qui calcule la distance euclidienne entre deux villes `city1` et `city2` (chacune étant représentée par un couple de coordonnées).

Question 4 - Écrire la fonction

```
def distance_matrix(cities):
```

qui calcule et renvoie la matrice des distances entre villes (`cities` une liste de coordonnées).

Question 5 - Écrire la fonction

```
def tour_length(tour, distances):
```

qui calcule la longueur d'un cycle (`tour`) en faisant simplement la somme des distances entre villes (ne pas oublier de "fermer" le cycle) à partir de la matrice de distances (`distances`) entre villes

2 Approche exhaustive

Une approche de résolution exacte (dite par force brute) consiste à tester tous les cycles hamiltoniens, puis à sélectionner celui qui a la plus petite longueur.

Calculer l'ensemble des cycles peut se faire simplement en générant les $n!$ permutations de la liste $(1, 2, 3, \ldots, n)$. Il est ensuite possible de calculer la longueur de chacun de ces cycles, puis de connaître le plus court d'entre eux.

> Ici, on calcule plus de cycles que nécessaire puisque certains sont équivalents $(1, 2, 3)$ et $(2, 3, 1)$ par exemple. Ce n'est pas très important pour ce TP et il n'est donc pas nécessaire de définir une fonction plus efficace ne générant un cycle qu'une seul fois. Mais on peut obtenir une fonction n fois plus efficace de manière simple en fixant la ville de départ et en ne permutant que les $n - 1$ villes restantes.

Question 6 - Écrire la fonction

```
def generate_all_tours(n):
```

qui calcule et renvoie la liste de tous les cycles de longueur n (ensemble de toutes les permutations d'un vecteur $(1, 2, \ldots, n)$.

Question 7 - Écrire la fonction

```
def force_brute(distances):
```

qui cherche parmi tous les cycles lequel est le plus court et le renvoie. Le paramètres `distances` correspond à la matrice des distances calculées entre chaque ville.

Question 8 - Tester la fonction pour des ensembles de 5 à 20 villes et mesurer le temps d'exécution pour chacune des expériences.

3 Estimation par échantillonnage

Rapidement (plus de 13 villes à peu près) la méthode exhaustive n'est plus utilisable, vue la complexité de l'algorithme. Dans ce cas, on peut chercher une solution approchée, sans garantie d'avoir trouvé la meilleure solution.

Une manière simple de procéder consiste à tirer aléatoirement un ensemble de cycles, puis de conserver le plus court. Plus la taille de l'échantillon est grand, plus on se rapproche de l'approche exhaustive.

Question 9 - Tester cette approche avec 10 villes (ce qui permet de comparer le résultat obtenu avec la solution exacte). Afficher la courbe de la longueur trouvée en fonction de la taille d'échantillon (voir exemple donné en figure 2).

Question 10 - Tester aussi cette approche avec 50 villes. Est-ce vraiment efficace ?

4 Approche heuristique

Lorsque le nombre de villes augmente encore, il peut être intéressant de chercher une heuristique pour construire une solution comme celle du plus proche voisin vue en M61.

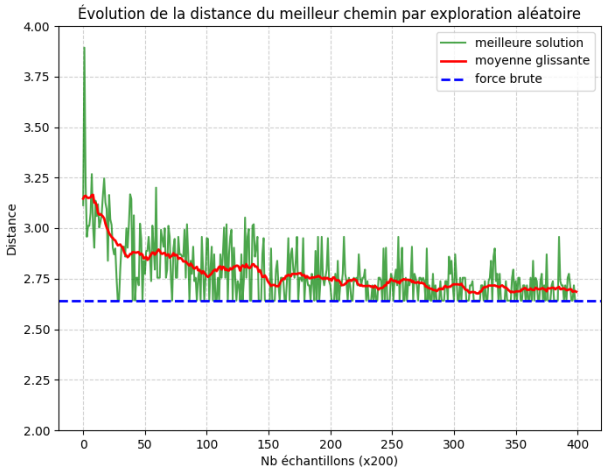

FIGURE 2 – Évolution de la longueur de la solution trouvée par l'exploration aléatoire (tirage avec remise) en fonction du nombre d'échantillons tirés (exemple sur 10 villes, meilleure solution exacte : 2.64). On remarque qu'avec 5000 échantillons la meilleure solution avait été trouvée.

Question 11 - Coder une fonction réalisant cette heuristique et la tester avec 10 villes pour comparer le résultat obtenu avec la solution exacte et l'approche précédente.

Question 12 - Tester la même approche avec 50 villes et comparer le résultat ainsi que le temps de calcul nécessaire avec l'estimation par échantillonnage.

5 Approche par algorithme génétique

Comme vu en cours, les algorithmes génétiques sont adaptés à ce type de problèmes et permettent d'obtenir efficacement une solution approchée à moindre coût. Pour la suite, on pourra s'appuyer sur l'article suivant (disponible sur Moodle) : Larranaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., & Dizdarevic, S. (1999). Genetic algorithms for the travelling salesman problem : A review of representations and operators. *Artificial intelligence review*, 13, 129-170.

L'algorithme complet à mettre en œuvre est décrit dans l'article sur la Figure 1 p.131. La fonction de fitness à optimiser est simplement la valeur du cycle le plus court de la population.

Population initiale

Question 13 - Écrire la fonction

```
def random_population(nb_tours, n):
```

qui génère une population aléatoire de `nb_tours` individus (cycles) de longueur `n`.

Mutation d'un individu

Question 14 - Écrire les fonctions de mutation d'un individu, selon les méthodes *displacement mutation* et *exchange mutation* (pp.148-149).

```
def displacement_mutation(tour):

def exchange_mutation(tour):
```

Croisement de deux parents

Question 15 - Écrire la fonction effectuant le croisement de deux individus, selon la méthode *cycle crossover* vue en TD (p. 5 du support M61 ou p.138 de l'article).

```
def cycle_crossover(parent1, parent2):
```

Question 16 - Écrire les fonctions alternatives effectuant le croisement de deux individus selon les méthodes *Partially-mapped crossover* (PMX) et *order crossover* (pp.138-140).

```
def pmx_crossover(parent1, parent2):

def order_crossover(parent1, parent2):
```

Algorithme génétique

Question 17 - Écrire une fonction implémentant un algorithme génétique utilisant ces fonctions selon l'algorithme suivant :

```
begin algo_gen
  génération de la population initiale : random_population(nb_tours, n)
  while nb_generations < MAX_GEN
  begin
    sélection de m parents depuis la population
    génération des enfants par croisement (cycle_crossover par exemple)
    mutation des individus de la population (displacement_mutation par exemple)
    ajout des enfants à la population
    réduction de la population en sélectionnant les nb_tours meilleurs individus
    incrémentation de nb_generations
  end
  affichage du meilleur individu
end algo_gen
```

Question 18 - Calculer le nombre de solutions évaluées par rapport à l'approche aléatoire pour une performance équivalente.

Question 19 - Comparer les performances en fonction de la fonction de mutation utilisée et du taux de mutation. De même pour les différentes fonctions de croisement.

Question 20 - Comparer la performance de l'algorithme génétique avec l'approche par plus proche voisin et par échantillonnage.